

Introduction to AI and ML

Project Report

Career Suggestion Expert System using Prolog

Name:

Aarush Sudheer

Registration Number:

25BAI10711

Type:

Project Report

Institution:

VIT University, Bhopal

1. Introduction

Career decisions are strange. They feel like they should be straightforward, like there should be some method for working out the right answer, but in practice most people I have observed, including myself, end up defaulting to something more like: what do people around me seem to be doing, and what does my family think is sensible. Actual reflection on interests and strengths tends to happen much later, if at all, and often after the initial choice has already been made.

This project started from that observation. The idea was to build a small expert system in Prolog that takes a few inputs from a user, what they are interested in, how skilled they feel, what kind of work appeals to them, and returns a set of career suggestions based on stored knowledge and logical rules. It is not a replacement for proper guidance. I want to be clear about that from the start. But it is a working demonstration of how rule-based reasoning can structure a decision that people typically navigate without any structure at all, and building it was a useful way to get into how expert systems actually function.

2. Problem Statement

The core problem is one that is fairly easy to observe: students frequently end up in careers or degree specializations that do not fit them particularly well, not because of bad luck, but because the decision was made without enough information or reflection. There are a few specific reasons this happens:

- Interests are often not well-defined early on, and people are not always good at identifying what they actually enjoy versus what they have just been told is valuable
- Skills and career requirements are hard to match without some structured framework, particularly when a student has no professional experience yet
- The guidance that does exist, from counselors, institutions, or career fairs, tends to be broad and generic rather than responsive to an individual's specific profile

The result is that career choice often becomes a process of elimination or imitation rather than genuine matching. A tool that makes the matching process more explicit, even in a simplified form, seemed like a worthwhile thing to build.

3. Objective

Going into this, I had three concrete goals and one informal one. The concrete goals: build a system that actually runs and returns useful output rather than just existing as a design document; make it work through logical inference rather than a simple lookup, meaning the system reasons toward conclusions rather than just pattern-matching text; and implement it in Prolog, which is the language this course introduced and which is genuinely well-suited to this kind of knowledge-representation problem.

The informal goal was to understand what I built well enough to explain it to someone else. That bar is higher than it sounds. A system you can run but cannot explain is not a system you really understand, and one of the things I wanted from this project was actual familiarity with how Prolog handles inference, not just a working file I could submit.

4. Proposed Solution

The system collects three inputs from the user: their interest area, their self-assessed skill level, and their preferred type of work. It then runs those inputs against a Prolog knowledge base that stores career data as facts, each career tagged with the interest area, skill level, and work type it fits best. Rules check whether a given career satisfies all three conditions for the user's input, and those that do get returned as suggestions.

The distinction that matters here, the one that makes this an expert system rather than just a filtered list, is that the system applies rules to draw conclusions rather than retrieving pre-written answers. A spreadsheet lookup returns rows that match a query. A rule-based system evaluates logical conditions and infers which conclusions follow. For a small project like this the practical

difference is modest, but the underlying mechanism is meaningfully different, and understanding that difference was one of the goals.

5. Tools and Technologies Used

The project used three tools, each serving a fairly distinct role:

- SWI-Prolog — the environment where the program was written and tested. SWI-Prolog is well-documented and widely used for this kind of logic programming work, which made it easier to find answers when I got stuck
- Visual Studio Code — used as the editor. There is nothing Prolog-specific about it, but the Prolog extension added syntax highlighting which helped a lot when trying to track down errors in clause structure
- GitHub — used to version-control the project throughout development. This turned out to be more useful than I expected, since being able to roll back to an earlier working version after a change broke something saved a fair amount of time

6. System Design

The design centers on three input dimensions that together describe what a user is looking for:

- Interest — the broad subject area the user finds engaging, with options covering coding, design, business, and research
- Skill level — the user's own assessment of where they currently stand, ranging from low to medium to high
- Work type — the kind of day-to-day activity the user prefers, either creative, analytical, or practical

Each career stored in the knowledge base is associated with one value for each of these dimensions. When all three of the user's inputs match a career's attributes, that career is returned as a suggestion. If multiple careers match, all of them are returned. If none match, the system tells the user rather than returning a silent empty result, which is a small but important design choice for usability.

The design is deliberately kept simple. A more complex weighting system or a probabilistic matching approach would produce more nuanced results, but it would also make the logic harder to follow. For the purposes of this project, transparency in the reasoning was a priority over sophistication in the output.

7. Features of the System

Beyond the core input-suggestion loop, the system includes a few features that make it more usable in practice:

- A step-by-step menu that walks the user through each input rather than expecting them to understand the interface from scratch — this keeps the interaction straightforward even for someone unfamiliar with terminal-based programs
- Input validation that catches anything outside the accepted options and loops back to ask again, so a typo or misread choice does not just break the query silently
- Support for multiple suggestions when several careers match the input, rather than arbitrarily returning just one
- A knowledge base structure that is easy to extend — adding a new career means adding a single new fact, and the rules do not need to change

None of these individually required much code. Together they push the system from a proof of concept into something that feels usable, which I think matters even for a coursework project.

8. Implementation

The implementation rests on three things Prolog is specifically designed to handle well.

Facts form the knowledge base. Each career is stored as a fact specifying its name and the three attribute values associated with it. This is the raw information the system reasons over.

Rules define the matching logic. A suggestion rule takes the user's three inputs and checks whether there exists a career fact in the knowledge base whose attributes match all three. Prolog's

unification mechanism handles the checking. If a match is found, Prolog succeeds and returns the career. If not, it fails and moves on.

Recursion handles the user-facing parts of the program. The menu loops until the user selects a valid option. The input prompts loop until valid input is received. Writing these loops in Prolog, which does not have traditional iterative constructs, meant using recursive predicates, and getting those right was one of the more instructive parts of the project.

The thing I found hardest to adjust to was Prolog's declarative style. In most languages I have used, you write a sequence of instructions: do this, then do that, then check this condition. In Prolog, you describe relationships and constraints, and the inference engine figures out what follows. The mental shift from telling the program what to do to telling it what is true took time, and there were several points early in development where I was writing Prolog as if it were Python, which does not work.

9. Challenges Faced

Prolog is less forgiving than most languages I had used before this project. The syntax rules are strict in ways that produce confusing errors if you do not already know what to look for. A clause that is missing its closing period, a variable written in lowercase when Prolog requires uppercase, a predicate name with a typo that does not match anything in the knowledge base — any of these produces an error whose message does not always point directly to the problem. Early development was a fairly slow cycle of running the program, reading the output, guessing at the cause, making a change, and trying again.

Input validation took longer than I expected. The basic challenge is that when a user types something invalid, the system needs to reject it, explain what the valid options are, and ask again, all without crashing or getting into an inconsistent state. In Prolog this means writing a recursive predicate that catches failure and re-invokes itself. Understanding why the predicate structure needed to work the way it did took a few attempts.

Getting the matching logic right required careful testing. The rule that suggests a career checks three conditions simultaneously, and there are combinations of inputs that should return

results and others that should not. Building a set of test cases and running through them manually to verify the outputs took a while, and it found a couple of edge cases where the logic was slightly off in ways that were not obvious from just reading the code.

10. Learning Outcomes

The most useful thing I took from this project was a working understanding of how expert systems function at the level of implementation, not just definition. Reading that an expert system uses a knowledge base and an inference engine is one thing. Writing the knowledge base, writing the rules, watching Prolog's inference engine succeed and fail across different inputs, debugging it when the inferences were wrong — that is a different kind of understanding, and it is the kind that actually sticks.

Prolog itself was a significant learning. The declarative paradigm is genuinely different from procedural or object-oriented programming, and I do not think I could have fully grasped that difference from reading about it. Having to actually build something in it, and having the places where my mental model was wrong show up as broken code, was the only way I was going to internalize how it works. I am not going to claim Prolog is now one of my stronger skills. But I understand it well enough now that I could read someone else's Prolog program and follow the logic, which is more than I could say at the start.

On the practical side: input validation, recursive control flow, and the structure of a rule-based inference system are all things I understand better now than I did before. And the exercise of translating a real-world decision process into formal logical rules was more interesting than I expected it to be. It surfaces assumptions that are easy to leave implicit in informal reasoning.

11. Future Improvements

The system works as designed, but the design is minimal. There are several obvious directions for improvement:

- The career knowledge base is small, which means many input combinations return no suggestions. Expanding it is the highest-priority improvement — it requires no changes to the core logic, just adding more facts
- A ranking or scoring system would make the output more useful. Currently all matching careers are returned equally. A system that identified closer and looser matches, and ordered the results accordingly, would be more informative
- A graphical or web-based interface would make the system accessible to people who are not comfortable running programs in a terminal. The logic would not need to change, but the front-end would need to be rebuilt around something like a simple web form
- Additional input dimensions would allow for more precise matching. Preferences around salary range, remote versus office work, or industry sector would all make the suggestions more relevant to a specific user

The Prolog architecture handles extensions reasonably well. New careers are new facts. New input dimensions require adding new parameters to the existing rules, but the overall structure stays the same. The system would need to be substantially redesigned only if the matching logic became significantly more complex than it currently is.

12. Conclusion

This project built what it set out to build: a working career suggestion expert system in Prolog that takes structured user input and returns logically matched career options. The system is not complex, and I am not going to overstate what it demonstrates. But it does what it says it does, and building it required understanding the parts well enough to debug them when they did not work.

What I found more interesting than I expected was how much the act of implementing something clarified concepts that had felt vague before. Expert systems, knowledge bases, inference rules, declarative programming — these are terms I had encountered before this project. After it, they mean something specific and concrete to me rather than just gesturing at a general area of AI. That shift is harder to produce by reading than by doing, which I think is the argument for project-based coursework in general.

Prolog is a narrow tool. It is well-suited to a specific category of problems and not well-suited to most others. But working within those constraints turned out to be instructive in a way

that a more flexible language might not have been, because it forced me to think carefully about logical structure rather than reaching for familiar patterns. I would rather have that understanding than not, even if I end up not writing much Prolog after this course.